AI-Based Smart Traffic Management

Using Satellite Data — Traffic Data Analysis

Intersection-Level Congestion Patterns: Kolkata & Dhaka

Problem Statement

Why cities need AI-based, satellite-informed traffic management

Traffic congestion in cities causes long travel times, fuel wastage, air pollution, and delays for emergency vehicles. Existing traffic management systems often rely on fixed signal timings or limited real-time information. Although navigation apps provide traffic updates, they do not optimize traffic signals.

There is a need for an AI-based traffic management system that uses satellite data (along with available traffic information) to analyze conditions, identify congestion, and support smarter signal decisions — improving traffic flow and reducing delays.

How this differs from Google Maps

- AI traffic prediction — forecast congestion 15–30 minutes in advance
- Traffic signal optimization — recommend green signal timings
- Emergency vehicle priority alerts for ambulances and fire trucks
- Accident hotspot analysis using historical data
- Traffic reports for city authorities with analytics and suggestions

Target Users

Who the platform is built for

Traffic Police Department

Real-time enforcement and signal decisions

Smart City Authorities

City-wide planning and infrastructure investment

Municipal / City Traffic Management

Day-to-day operations and monitoring

Emergency Services

Ambulance and fire department priority routing

Daily Road Users & Commuters

Better-informed travel decisions

Dataset Overview

20 intersections across 2 cities, measured for vehicle volume and congestion level

2

Cities

Kolkata & Dhaka

20

Intersections

10 per city

2

Time Periods

Peak & Off-Peak

3

Traffic Levels

Low / Medium / High

Fields Captured

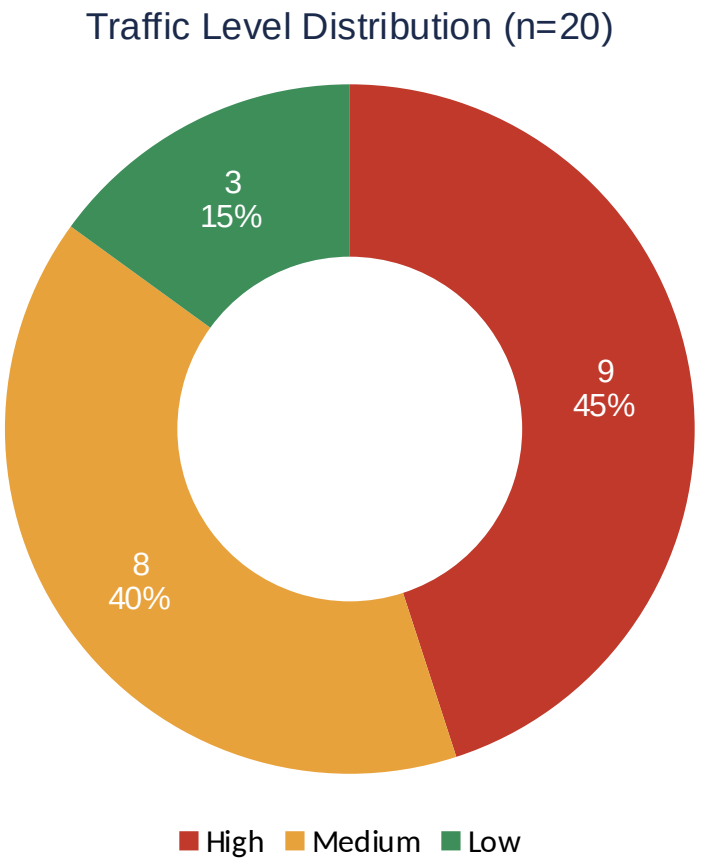
- City & Intersection ID
- Number of Roads meeting at the intersection
- Number of Lanes
- Vehicles Observed (count)
- Traffic Level (Low / Medium / High)
- Time Period (Peak / Off-Peak)

Why this matters

This ground/sensor-level data mirrors what satellite and camera feeds would report in the proposed AI system — vehicle density per lane, per intersection, split by time of day. It's the kind of input the prediction and signal-optimization models would train on.

Congestion Levels Across All Intersections

9 of 20 intersections (45%) already run at High congestion



Reading the split

High congestion (45%)

9 intersections — every single one occurs during Peak periods.

Medium congestion (40%)

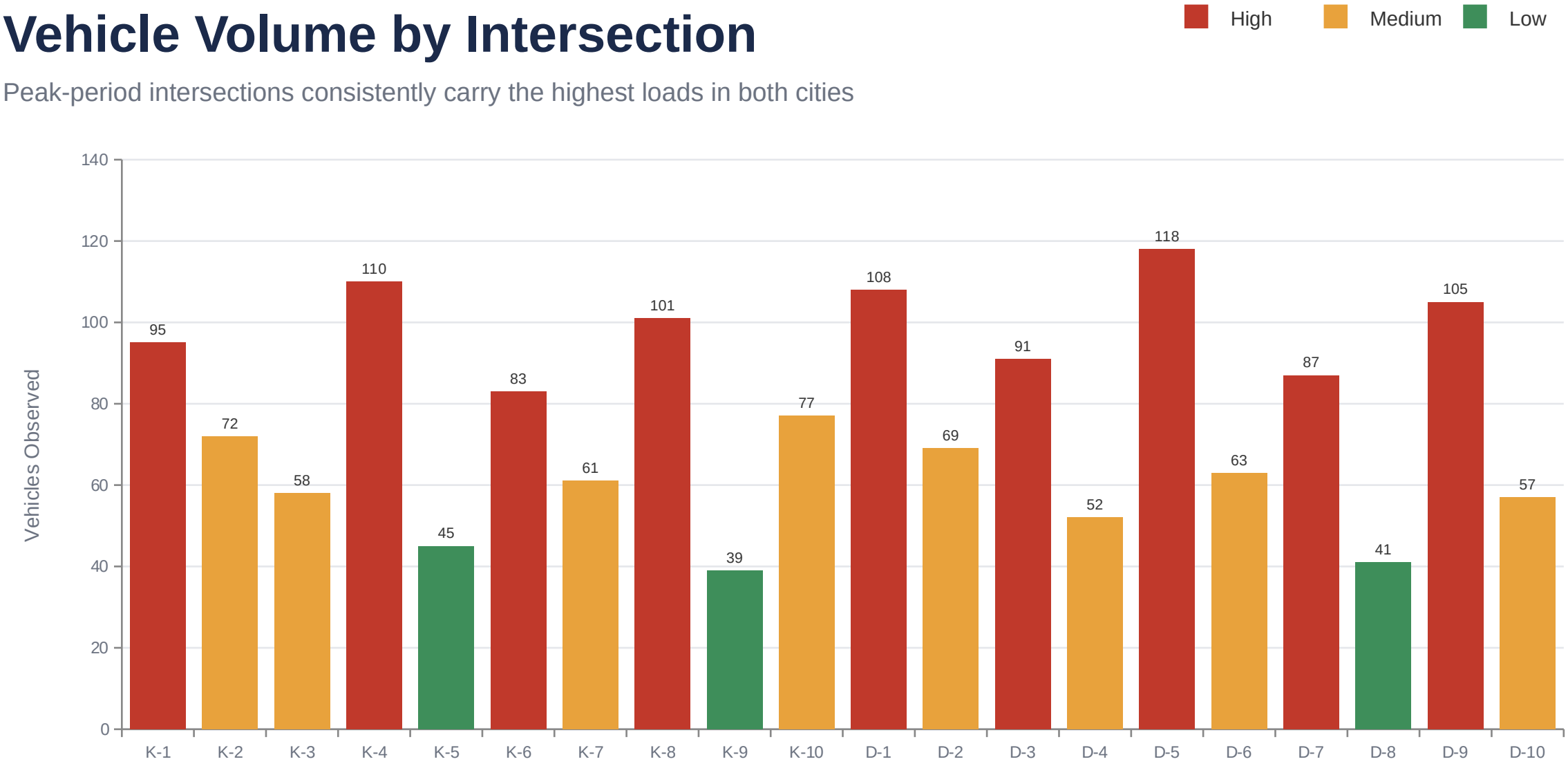
8 intersections — split between Peak (2) and Off-Peak (6).

Low congestion (15%)

Only 3 intersections, all during Off-Peak.

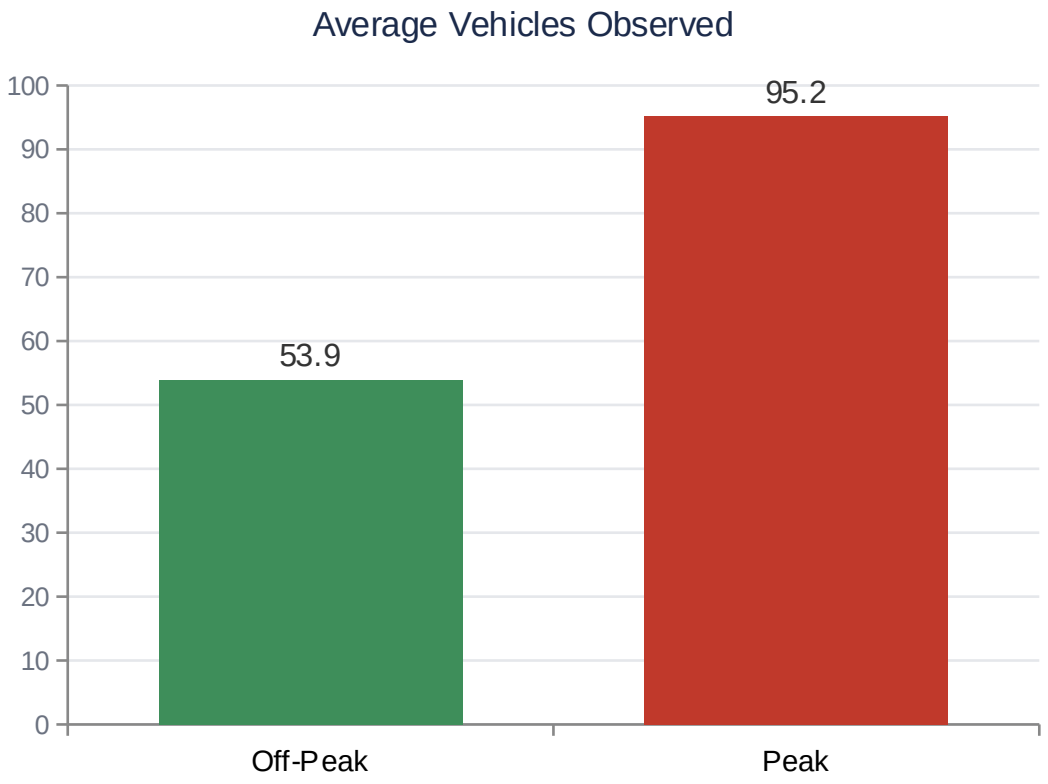
Vehicle Volume by Intersection

Peak-period intersections consistently carry the highest loads in both cities



Peak vs. Off-Peak Comparison

Peak-period volume is 77% higher on average, with zero Low-congestion readings



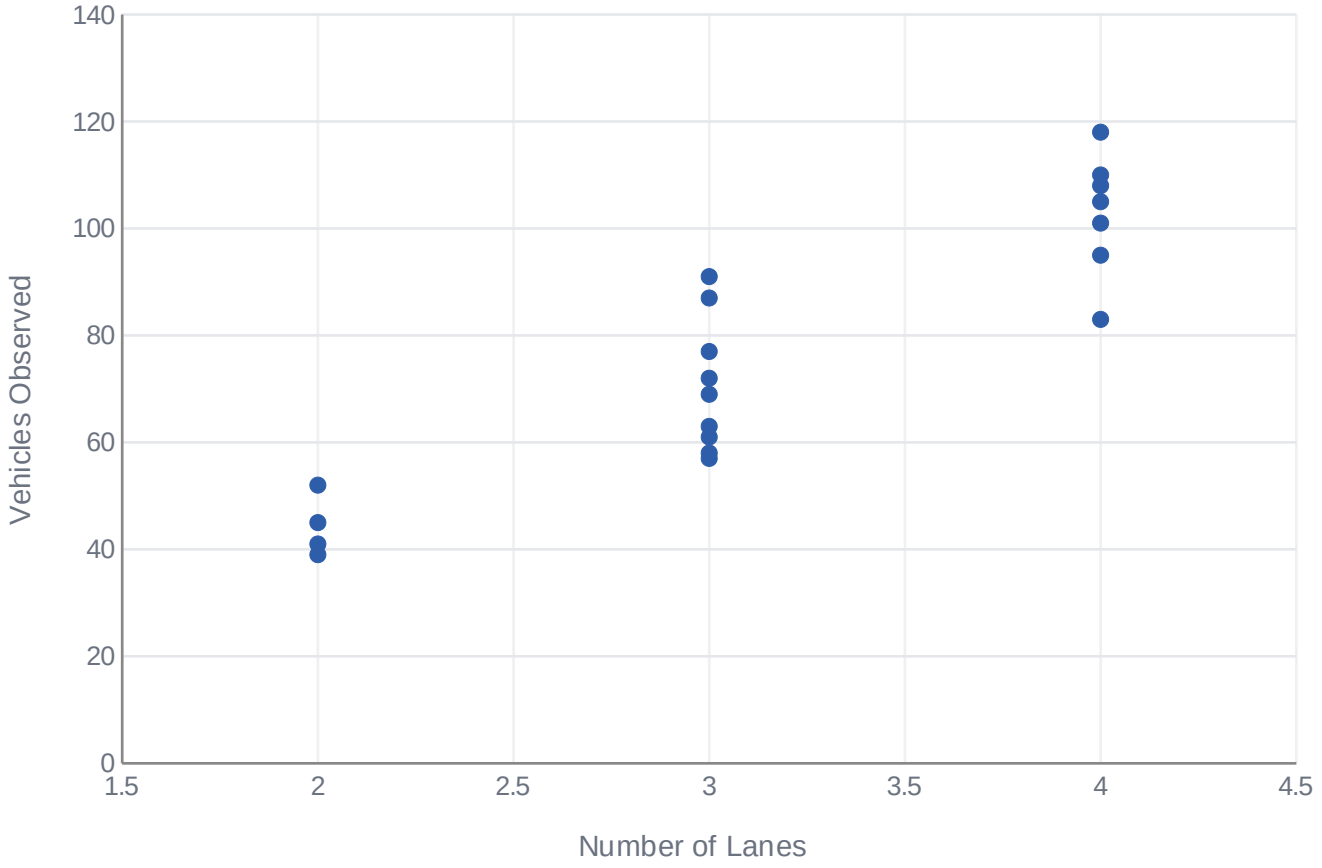
Congestion level by time period

	Peak	Off-Peak
High	9	0
Medium	2	6
Low	0	3

Takeaway: every High-congestion reading in the dataset occurs during Peak hours — this is exactly the window where AI-driven signal optimization and predictive alerts would have the most impact.

Lane Capacity vs. Vehicle Volume

Strong positive correlation (r = 0.92) between number of lanes and vehicles observed



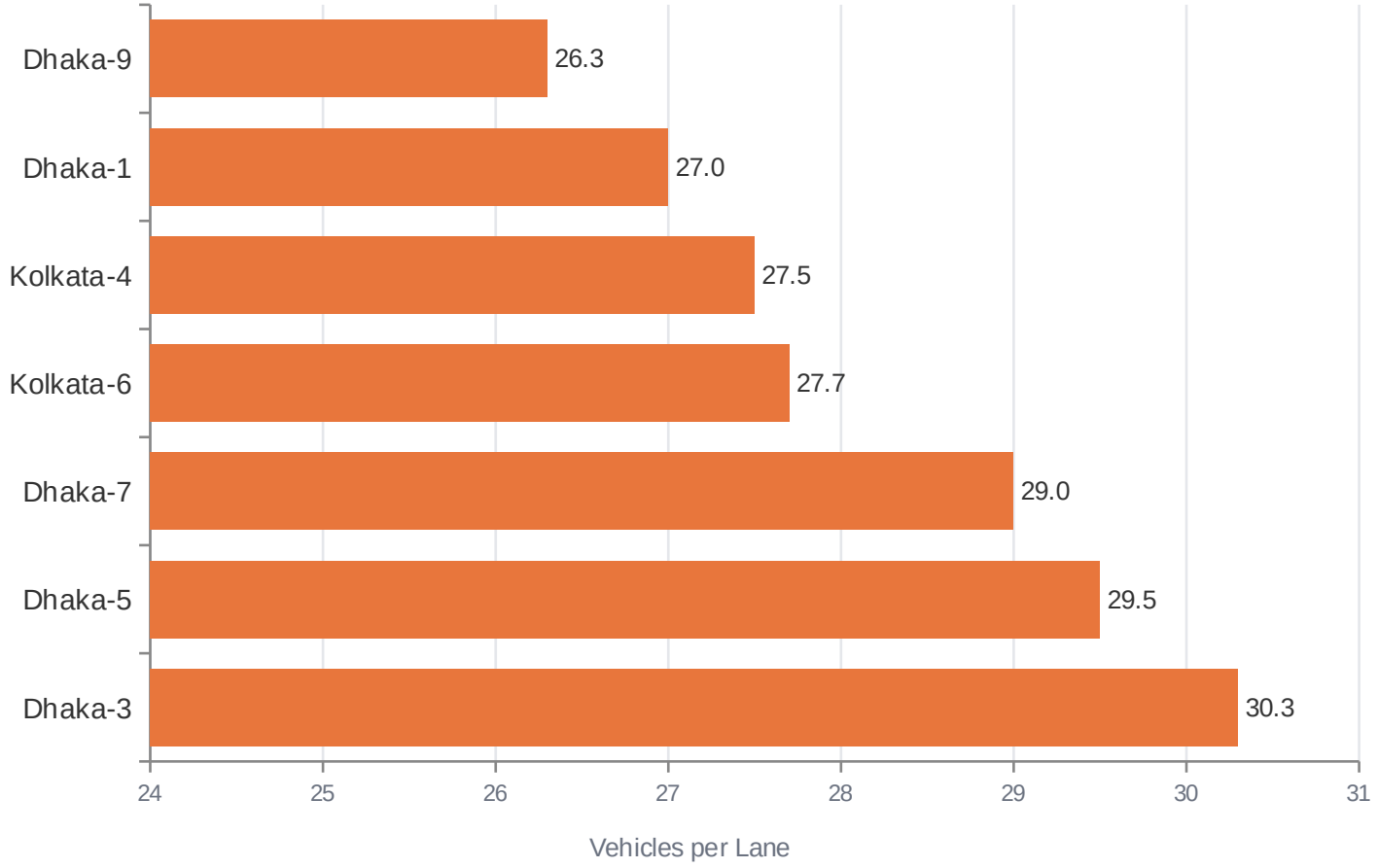
Correlation findings

- Lanes ↔ Vehicles: r = 0.92 (very strong)
- Roads ↔ Vehicles: r = 0.86 (strong)
- 4-lane intersections average 106 vehicles vs. 44 for 2-lane

Implication: raw lane count alone doesn't predict congestion at Off-Peak — this is where satellite-fed, real-time AI prediction adds value over static infrastructure metrics.

Top Congestion Hotspots (Vehicles per Lane)

Dhaka Intersections 3, 5 & 7 show the heaviest per-lane load



Why this metric matters

Vehicles-per-lane normalizes for road width, exposing the intersections under the most real strain — not just the busiest in absolute terms. These 7 locations are the strongest early candidates for AI-driven adaptive signal timing.

All 7 top hotspots are Peak-period, High-congestion intersections — confirming Peak hours as the priority deployment window.

Key Insights

1 Peak hours drive congestion

100% of High-traffic readings occur during Peak periods — off-peak sees zero High readings.

2 Lane count strongly predicts load

r = 0.92 correlation between lanes and vehicles observed; 4-lane intersections carry ~2.4x the volume of 2-lane ones.

3 Dhaka runs slightly hotter

Dhaka's average vehicle count (79.1) edges out Kolkata's (74.1), with 5 High-congestion intersections vs. 4.

4 A focused set of hotspots exists

7 intersections (mostly Dhaka) show the highest vehicles-per-lane — ideal first targets for adaptive signal AI.

Expected Dashboard

Core modules the AI platform surfaces to each user group

Live Congestion Map

Satellite + sensor-fed view of current traffic level by intersection, color-coded Low/Medium/High.

15–30 Min Prediction

Forecast panel showing which intersections are trending toward congestion.

Signal Timing Recommendations

Suggested green-light durations per intersection, updated as conditions change.

Emergency Vehicle Priority

Alerts and auto-suggested signal overrides along ambulance/fire routes.

Accident Hotspot Analysis

Historical overlay flagging intersections with repeat incidents.

Authority Reports

Exportable analytics for city authorities — trends, peak-hour summaries, hotspot rankings.

Python Analysis Code

Code used to generate the statistics and charts in this deck (pandas)

```
import pandas as pd

df = pd.read_excel('traffic_analysis_dataset.xlsx')

# Traffic level distribution
print(df['Traffic_Level'].value_counts())

# Average vehicles by city and by time period
print(df.groupby('City')['Vehicles_Observed'].mean())
print(df.groupby('Time_Period')['Vehicles_Observed'].mean())

# Congestion level vs. time period
print(pd.crosstab(df['Time_Period'], df['Traffic_Level']))

# Correlation: roads / lanes vs. vehicle volume
cols = ['Number_of_Roads','Number_of_Lanes','Vehicles_Observed']
print(df[cols].corr())

# Hotspots: vehicles per lane, top 7
df['Vehicles_per_Lane'] = df['Vehicles_Observed'] / df['Number_of_Lanes']
top7 = df.sort_values('Vehicles_per_Lane', ascending=False).head(7)
print(top7[['City','Intersection','Vehicles_per_Lane']])
```

Recommendations for the AI System

How these findings inform the proposed satellite-based traffic management platform

1

Prioritize Peak-hour prediction

Train the 15–30 min congestion forecast model primarily on Peak-window patterns, where nearly all High-traffic events occur.

2

Target high-load intersections first

Pilot adaptive signal optimization at the 7 identified hotspots (vehicles/lane) before city-wide rollout.

3

Use lane count as a baseline feature

Given r = 0.92 with volume, lane/road counts are strong low-cost predictors to combine with live satellite feeds.

4

Expand data collection

20 intersections is a proof-of-concept scale; broaden to more cities and intersections, and add weather/event/accident data.

Python Analysis Code

traffic_analysis.py — full script used to generate this deck's statistics and charts

```
"""
AI-Based Smart Traffic Management — Traffic Data Analysis
Analyzes intersection-level traffic data for Kolkata & Dhaka.
"""

import pandas as pd

# ---- Load data ----
df = pd.read_excel('traffic_analysis_dataset.xlsx')

print("=== Raw Data ===")
print(df.to_string())

# ---- Traffic level distribution ----
print("\n=== Traffic Level Distribution ===")
print(df['Traffic_Level'].value_counts())

# ---- Average vehicles by city ----
print("\n=== Average Vehicles Observed by City ===")
print(df.groupby('City')['Vehicles_Observed'].agg(['mean', 'min', 'max']))
```

Python Analysis Code (cont'd)

traffic_analysis.py — full script used to generate this deck's statistics and charts

```
# ---- Average vehicles by time period ----
print("\n=== Average Vehicles Observed by Time Period ===")
print(df.groupby('Time_Period')['Vehicles_Observed'].agg(['mean', 'min', 'max']))

# ---- Congestion level vs. time period ----
print("\n=== Traffic Level by Time Period ===")
print(pd.crosstab(df['Time_Period'], df['Traffic_Level']))

# ---- Congestion level vs. city ----
print("\n=== Traffic Level by City ===")
print(pd.crosstab(df['City'], df['Traffic_Level']))

# ---- Correlation: roads / lanes vs. vehicle volume ----
print("\n=== Correlation Matrix ===")
cols = ['Number_of_Roads', 'Number_of_Lanes', 'Vehicles_Observed']
print(df[cols].corr())

# ---- Hotspots: vehicles per lane ----
df['Vehicles_per_Lane'] = df['Vehicles_Observed'] / df['Number_of_Lanes']
```

Python Analysis Code (cont'd)

traffic_analysis.py — full script used to generate this deck's statistics and charts

```
print("\n=== Top 7 Congestion Hotspots (Vehicles per Lane) ===")
top7 = df.sort_values('Vehicles_per_Lane', ascending=False).head(7)
print(top7[['City', 'Intersection', 'Vehicles_per_Lane']].to_string(index=False))
```